POKEMONAI · RESEARCH REPORT AND DECISION GUIDE

From Hand-Tuned Weights to a Self-Training Agent

how the Strategy agent should learn, once manual blunder correction stops scaling

---

A synthesis of 21 primary sources on self-play reinforcement learning, per-decision credit assignment, interpretable hybrid policies, metagame adaptation, and evaluation under heavy luck — mapped onto this repository's constraints, with a selection framework and a staged build plan.

Deep-research run of 2026-07-11 · 103 claims extracted · 25 adversarially verified

Companion to `docs/research/ml-training-system.md`

How to read this report

This document answers one question: **what should the training system for this agent become, and how do we choose?** It is written to be argued with. Every factual claim is tagged with its evidence status, every candidate path gets its strongest case before its verdict, and the final recommendation comes with explicit triggers that would change it.

Two evidence tags recur throughout:

VERIFIED. the claim survived a three-vote adversarial verification pass — three independent agents each tried to refute it against the primary source, and at most one succeeded. These are load-bearing.

EXTRACTED. the claim was pulled from a fetched primary source with a verbatim supporting quote, but fell outside the verification budget (the top 25 of 103 claims were verified). Strong leads; re-verify before betting the architecture on one alone.

The coloured blocks follow the textbook's convention: green blocks carry the **evidence**, orange blocks describe **roads considered and not taken**, violet blocks translate a finding into **this repository's** terms, and blue blocks mark the **key ideas** the whole argument hangs on.

Sections 1–2 set up the problem and the verified core of the evidence. Section 3 walks the six candidate paths in full. Section 4 maps the evidence onto the five demands the training system must meet. Section 5 is the selection framework — criteria, scoring, and the sensitivity analysis. Sections 6–8 give the staged build plan, its risks, and the open questions. Section 9 is the source annex.

Contents

1 The wall we hit

The agent today is roughly 95 percent hand-authored decision machinery. A linear scoring layer — hundreds of weighted Hypotheses, each a human-readable rule with an initial weight — ranks every legal option; a Turn Planner sequences the turn; an exact lethal solver owns the win rung; the Read recognises the opponent's archetype and swaps in matchup Briefs and weight overrides. Weights are tuned from **corrections**: manually reviewed misplays, harvested one blunder round at a time, fed through a perceptron-style update in `tools/train/tune.py`.

This built a genuinely strong agent, and the interpretability is not decoration — the Kaggle Strategy category judges the decision-making approach itself. But the effort model has a wall in it, and we have now hit it from three directions at once:

1. **Per-deck cost.** Every new deck needs a doctrine (deck-genie), a blunder round or three, and a correction-review cycle. The 2026-07-09 to 07-11 rounds each consumed a working session to produce seven to twelve reviewed items — for decks we already understood.
2. **Per-matchup cost.** Weights are matchup-blind unless somebody authors an override. Each meta shift adds opponent archetypes multiplicatively: the work scales as decks × matchups, and briefs go stale the day the meta moves.
3. **Per-rotation cost.** A standard rotation invalidates card facts, doctrines, briefs, and an unknown fraction of the weight mass simultaneously. Today the recovery plan is "do all of the manual work again, faster."

The question is not whether to add machine learning — a value model (Tier 5) already exists, gated off. The question is **which** learning system turns match experience into better decisions with the smallest human loop, without giving up the interpretable layer that is both our competition deliverable and our debugging surface.

> **KEY IDEA**
> The demands on that system, in the words of the original brief: it must fine-tune weights **context-aware**; it must be able to **split a feature** when a novel board state or matchup demands different behaviour; it must recognise that an agent **played every decision well and still lost** (and the converse); it must know **when to take risks** and when to develop; and it must stay competitive **as the standard and meta progress** with minimal manual intervention. Hundreds or thousands of manually created features will never get us there.

One more local fact shapes everything downstream: we have already learned, expensively, that cross-deck round-robin gauntlet winrates were uninformative for measuring improvements. Any proposed training loop that cannot **measure** its own progress is dead on arrival, so evaluation methodology is treated here as a first-class design problem, not an afterthought.

Our assets, stated plainly, because the selection framework leans on them: a fast native simulator (about 1,000 games per minute on one desktop — call it 1.4 million games per day), a pure-Python engine twin (`src/cgpy/`, ADR-0050) that gives us seed control and perfect-information instrumentation, an existing telemetry spine (`live_trace`), an existing corrections pipeline with a reviewed ledger, and a CPU-only, offline, ten-minutes-per-match grader on the other end.

2 The verified core: eight findings

Everything in this section survived adversarial verification. These are the fixed points the rest of the report reasons from.

2.1 A desktop-budget existence proof exists — [VERIFIED]

DouZero mastered DouDizhu — a large, shuffled-deck, hidden-hand card game with a massive action space whose legality changes every turn — **from scratch on one server with four consumer GPUs, in days of training**, and ranked first of 344 agents on the external Botzone leaderboard. The method is almost embarrassingly simple: Deep Monte-Carlo, meaning sampled final-outcome returns regressed by a neural network, with parallel actors generating games.

Two design details carry most of the transfer value. First, **legal actions are encoded as inputs to the network, not as a fixed output head**: the network scores (state, action) pairs, so a turn-varying action menu costs nothing. The paper is explicit that standard policy-gradient and DQN methods failed on this game **because** of the turn-varying action space, and the PPO-based successor PerfectDou kept the action-as-input encoding. Second, the approach needed no search at training time — the simulator's throughput was the whole engine.

> **FOR THIS REPO Why this matters here**
> Our engine's select options are already structured objects — a scored (state, action) interface is exactly how the Pilot's Hypothesis layer works today. And our simulator throughput is the same order of magnitude DouZero fed on. The existence proof says: a **value function of professional quality** is trainable on hardware we own, provided we ask it the DouZero-shaped question (score this action in this state) rather than the fixed-action-head question.

2.2 "Played perfectly and lost" is a solved problem — [VERIFIED]

Suphx, Microsoft Research's Mahjong agent, is rated above 99.99 percent of officially ranked human players on the Tenhou platform — the first program above most top humans in a game so luck-dominated that a round's score is nearly uncorrelated with the quality of the round's decisions. Its central trick is the **global reward predictor**: a network Φ trained to predict final-game outcome from the state so far. During reinforcement learning, a round's reward is not its raw score but the **delta** $\Phi(x^k) - \Phi(x^{k-1})$ — how much the round changed the predicted final outcome. The paper's own motivating line: a negative round score may not mean a poor policy; it can reflect correct tactics.

> **KEY IDEA**
> Per-decision credit under heavy luck comes from a **learned value function's deltas**, never from raw outcomes. A lost match whose decisions never dropped predicted win probability was **played well** — the system can say so mechanically. A won match

containing a decision that cratered predicted win probability **contains a blunder** — the system flags it even though the scoreboard says victory. This single mechanism is the automated replacement for manual blunder labeling.

One caveat travels with this finding: Suphx trained Φ on top-human game logs. We have no human corpus for this card pool, so our analogue trains on self-play logs — the implications are handled in Section 6 (Stage 1) and Section 7 (risk 1).

2.3 Variance is the enemy; learned baselines are the weapon — [VERIFIED]

The counterfactual-regret-minimisation literature spent a decade converging on the same lesson from the theory side. Monte-Carlo CFR converges slowly **because** its sampled value estimates have high variance. VR-MCCFR reformulates the estimates around state-action **baselines** — exactly the control-variate idea from policy-gradient RL — and stays unbiased while cutting empirical variance by three orders of magnitude and convergence time by one; with a perfect baseline, the variance is provably zero. DREAM scaled this to neural networks with a learned history baseline but kept a high-variance importance-sampling term; ESCHER removed importance sampling entirely by learning a **history value function**, is unbiased, converges to approximate Nash with high probability, and measures regret-estimate variance eight to nine orders of magnitude below DREAM on benchmark games.

The read-across is not "run ESCHER." It is: **whatever per-decision learning signal we build must be baseline-corrected by a learned value function** — which is the same artifact as the blunder labeler above. Theory and practice point at the identical keystone.

2.4 Winning agents can still be trivially exploitable — [VERIFIED]

ByteDance's search-free self-play system beat a Hearthstone player whose peak rank was top-10 in China's official league, in all four best-of-five tournaments. The same lineage's ByteRL won the 2022 Legends of Code and Magic competition with an 84.41 percent winrate. And then independent researchers (including a DeepStack co-author) beat ByteRL **90.4 percent of the time** on 32-deck pools with an almost insultingly cheap pipeline: behavior-clone ByteRL's own self-play games (about 125 thousand matches, 3.5 million state-action pairs — that clone alone reached 42.4 percent), then fine-tune the clone with PPO as a targeted best-responder.

FOR THIS REPO The standing alarm we are missing

Ladder winrate certifies nothing about robustness. The literature's answer is a **standing exploiter probe**: periodically train a best-responder against the current agent and watch its winrate. Ours is cheap to build — the exploit recipe needs only our own self-play logs, which Stage 1 already collects. And note the dual use: Kaggle ladder replays of **opponents** are exactly the corpus for behavior-cloning opponent models.

2.5 The compute wall is real — and method-shaped — [VERIFIED]

The numbers, because they discipline every choice in Section 3: ByteRL's best non-cheating Hearthstone model trained on **24 V100 GPUs plus 5,856 CPU cores for 23 days**, consuming 320 million samples per learning period. The LOCM champion took 24 GPUs times 72 hours and billions of episodes. Student of Games used TPU fleets comparable to AlphaZero (order 3,500 TPUv4 actors). Meanwhile, a single-desktop PPO self-play pipeline — 100 thousand episodes on a GTX 1050 Ti — **plateaued around 50 to 59 percent** against a mid-level one-step-lookahead baseline, nowhere near the 90-plus percent of the large-compute systems.

But the wall is method-shaped, not absolute: DouZero reached its result on four consumer GPUs, and the desktop-PPO authors themselves attribute part of their gap to budget-forced choices (one-layer MLPs, no permutation-invariant encoding). The conclusion is not "desktops cannot learn card games." It is: **on a desktop, do not spend compute training an end-to-end neural policy; spend it training a value function and credit signals over an existing strong policy.** That is precisely the asset split we have.

2.6 Our gauntlet failure generalises — and has a fix — [VERIFIED]

The Hearthstone team's internal winrate table said cheat-model c5 beats non-cheat b4 head-to-head at 55 percent. Against the human, c5's checkpoint played **worse** — including an obvious blunder a human would never make (healing at full health). The PyTAG benchmark authors report the same thing generically: agent-vs-agent winrates give little insight into real strength. Our hard-won local lesson — cross-deck gauntlet winrates proved uninformative — is independently confirmed, and it has a mechanism (Section 3 of the AlphaStar finding: populations of strategies contain **millions** of rock-paper-scissors cycles; pairwise winrates are non-transitive at scale).

The same paper shows what the serious version costs: their machine-vs-machine protocol ran **45,000 matches per model pairing**, balanced over hero matchup (3×3) and first/second position (2×), before quoting a winrate. Raw high-N with balancing is table stakes; Section 5 of this report adds the variance-reduction machinery that buys the N back down.

2.7 Sound search is the right shape and the wrong price — [VERIFIED, one refutation]

Student of Games unified AlphaZero-style search with counterfactual-regret reasoning: grow a public-state search tree at decision time, alternate regret updates with expansion, back the leaves with a learned counterfactual value-and-policy network. It is provably sound (converges to Nash in two-player zero-sum as compute grows) and beat the best open poker bot and the Scotland Yard state of the art. Its declared bottleneck: it **enumerates all information states within each public state** — 1,326 possible hands in heads-up poker, versus something like 10^7–10^8 consistent opponent hand/deck states in a TCG.

The verification pass killed one claim here, and the kill matters: the assertion that this makes poker-style search **categorically unusable** for collectible card games was **refuted zero votes to three**. The SoG authors themselves propose generative sampling of world

states as the workaround, and follow-up work on history filtering pursues exactly that. Sound decision-time search over a **sampled belief set** is expensive today, not impossible — it is the long-term upgrade path for our Turn Planner, not the foundation to build first.

2.8 Hand-authored dense rewards actively hurt — [VERIFIED, medium confidence]

In LOCM, potential-based reward shaping — dense per-move rewards from opponent health loss, or from a competition-winning agent's heuristic score — failed to beat the sparse terminal win/loss signal, and the health-based variant was **statistically significantly worse** during stretches of training. The unshaped baseline produced the best agent. One study, on a deterministic simplified CCG, with tiny networks — hence medium confidence — but it points the same direction as Suphx and the CFR line: dense credit must come from **learned** value functions, never from hand-picked heuristic potentials. Do not wire "damage dealt this turn" into a reward channel, ever.

3 Six candidate paths, given their best case

This section is the juice: each path the literature offers, argued **for** as strongly as the evidence allows, then given its verdict against our constraints. The constraint set, as a reminder: desktop training compute; CPU-only ten-minute-match inference; no internet at runtime; interpretability as a scored deliverable; a fast simulator; and a hard requirement that per-new-deck manual effort collapse toward doctrine-authoring only.

3.1 Path A — End-to-end neural policy via self-play

The case for it. This is the only path with a verified top-human TCG scalp: the ByteDance system beat a peak-top-10 Hearthstone player across full games including deck-building, with no search at training or inference and no expert data. Its engineering lessons are concrete and portable: optimistic smooth fictitious play rather than naive self-play; V-trace with an UPGO auxiliary loss; discount $\gamma = 1.0$ on the terminal win/loss signal (worth plus seven points of winrate in their ablations); halving actor count to keep off-policy staleness near on-policy (plus ten points); splitting one shared policy into per-hero models (plus 6.5 points). If compute were free, this is the known-good recipe.

The case against it. Compute is not free: 24 V100s and nearly six thousand CPU cores for 23 days, against our one desktop — and the desktop-scale attempt of the same idea plateaued at 50 to 59 percent against a mid-tier baseline. The product is also exploitable (ByteRL, 90 percent, by a hobby-budget best-responder) and opaque: an end-to-end policy erases the interpretable layer that is both our writeup and our debugging story. In the Strategy category, "we ran PPO for three weeks" is a weak deliverable even if it worked.

> **ROAD NOT TAKEN Verdict**
>
> **Rejected as the backbone; strip it for parts.** Adopt: action-as-input encoding, $\gamma = 1.0$, staleness discipline, per-context model splitting, fictitious-play-style population training. Reject: the end-to-end neural policy itself, on compute, exploitability, and interpretability grounds simultaneously.

3.2 Path B — Regret minimisation (Deep CFR → DREAM → ESCHER)

The case for it. This is the theory-first path: equilibrium guarantees, principled handling of hidden information, and the newest member (ESCHER) is unbiased, model-free, needs no importance sampling, and wins over 90 percent head-to-head against its predecessors in dark chess — a genuinely large imperfect-information game. If we wanted an agent that is provably hard to exploit, this family is where such proofs live. Its variance analysis is also, quietly, the best theory we have for **why** our manual corrections were so noisy a signal.

The case against it. Every strict guarantee attaches to the tabular-with-oracle-value version; the deep variant inherits value-network approximation error. The variance measurements were taken on small benchmark games with oracle value functions, not TCG-scale domains — nobody has published ESCHER at our game size on our budget. And an

equilibrium policy is a **mixed** strategy that optimises for unexploitability, not for punishing the specific, flawed opponents on a ladder. We would be paying the family's full complexity bill for a guarantee that is not even quite the objective we want.

> **ROAD NOT TAKEN Verdict**
> **Adopt the lesson, not the algorithm.** The baseline-corrected, low-variance credit signal is non-negotiable and appears in Stage 2. Running an actual deep-CFR variant end-to-end is an open research question at our scale (Section 8, question 2) — revisit if the hybrid path stalls.

3.3 Path C — Sound decision-time search (DeepStack / Modicum / Student of Games)

The case for it. The inference economics are startlingly good. Modicum played master-level heads-up no-limit poker **in real time on a four-core CPU with 16 GB of RAM**, and its whole strategy cost 700 core-hours to compute — versus millions of core-hours for its abstraction-based predecessors. DeepStack's continual re-solving ran at 2.3 seconds median per action on one GTX 1080. The key inventions are small and elegant: depth-limited solving works if the opponent gets a **choice among a handful of continuation strategies** at the leaf (fewer than ten sufficed in poker; a single leaf value provably fails), and a tiny two-layer, 64-node value network was enough to back the leaves. Abstraction-based precomputed policies, by contrast, are declared "mostly obsolete" — local best response demolished all of them and could not touch the search-based agent. And our agent **already is** a searcher: the Turn Planner plus lethal solver is a depth-limited solver without the equilibrium machinery.

The case against it. The public-state enumeration bottleneck (Section 2.7): poker's 1,326 hands versus our 10^7-plus consistent hidden states. The sampled-belief workaround is real but research-grade — SoG's authors **propose** it; nobody has shipped it for a TCG. Full GT-CFR training is also TPU-fleet territory.

> **ROAD NOT TAKEN Verdict**
> **The upgrade path, not the starting point.** Phase later: once the Stage 1 value net exists, the planner can consume it as a leaf evaluator over a **sampled** set of opponent hands (our deck tracker and deck-content odds machinery already produce exactly the posterior to sample from). That is the SoG shape at hobby scale — and note Modicum's numbers say the inference budget fits our grader with room to spare.

3.4 Path D — Deep Monte-Carlo value learning (the DouZero recipe)

The case for it. Verified desktop-scale success on the most TCG-like game in the evidence set; conceptually the simplest thing that can possibly work (regress sampled final outcomes); thrives on exactly the asset we have in surplus (simulator throughput); and its action-as-input framing drops onto our option-scoring architecture without violence. Note what we need from it is **less** than what DouZero needed: DouZero had to extract the entire

policy from the value function; we only need the value function itself, because the policy layer — Pilot, planner, solver — already exists and is already good.

The case against it. Sampled-return targets are the highest-variance estimator in the toolbox (Section 2.3's whole point), so it needs volume — which we have — and eventually TD-style bootstrapping to sharpen. On its own it says nothing about interpretability, exploitability, or the meta; it is an engine, not a vehicle.

> **ROAD NOT TAKEN Verdict**
> **Adopted — as the training method for the keystone value network** (Stage 1), not as a standalone agent. This is the one path whose compute claim is verified at our budget.

3.5 Path E — Imitation and offline RL from replays

The case for it. The nearest-domain result in the entire evidence set: Metamon (RLC 2025) reconstructed over 475 thousand human Pokémon Showdown battles from third-person spectator logs into first-person trajectories — **no manual labeling anywhere** — then ran imitation learning, offline RL, and self-play fine-tuning through a transformer policy, and reached the top 10 percent of active human ladder players with no search at inference. Performance scaled with data (three million trajectories helped). The sequence model swallowed imperfect information whole: conditioning on the entire battle so far replaced explicit belief-state engineering. This is the strongest evidence that **replay corpora are training gold that requires no human annotation**.

The case against it. It is built on a resource we do not have: a deep public pool of human expert games for our exact card pool. Our replay sources are our own self-play (covered by Path D) and Kaggle ladder matches against other competitors — valuable, but thousands of games, not hundreds of thousands, and of unknown quality.

> **ROAD NOT TAKEN Verdict**
> **A data channel, not the backbone.** Harvest Kaggle ladder replays (the ADR-0019 access asymmetry already maps what we can pull) for three uses: behavior-cloned **opponent models** for the league (Stage 4), imitation **seeds** for exploiter probes, and a distribution-shift check for the value net (Section 7, risk 3). If the ladder ever exposes deep replay pools of top agents, this path's stock rises sharply.

3.6 Path F — The hybrid: keep the rule layer, learn the value and the corrections

The case for it. Every component has a literature anchor. Residual Policy Learning [EXTRACTED] shows model-free learning of a **corrective residual on top of a nondifferentiable hand-designed controller** — no gradients through the base needed — solving sparse-reward tasks pure RL fails, and composing with planner-style bases like our Turn Planner. Soemers, Piette and Browne [EXTRACTED] train **exactly our architecture** — a linear feature-weight policy — with zero manual labels via expert iteration: search visit-counts are the per-move target, cross-entropy SGD replaces the perceptron, and,

remarkably, **the feature set grows itself**: new compound features are minted by combining co-active feature pairs whose activation best correlates with the apprentice-vs-expert error, redundancy-penalised. That is a mechanised answer to "the ML must be able to consider splitting features." The LOCM evolution study [EXTRACTED] found a **linear** weight vector over about twenty hand-crafted features beat two more expressive genetic-programming tree representations under limited compute (54.8 versus 45.6 percent), and self-play fitness beat fixed-opponent fitness — the representation we already have is the right one for our budget. VIPER and INTERPRETER [EXTRACTED] close the loop from the other side: if a neural policy ever outgrows the rules, it distills back into small editable trees — VIPER's Q-loss-weighted sampling concentrates tree capacity on **critical states** (an order of magnitude smaller trees at equal strength), INTERPRETER's distilled Python programs ran at 79 microseconds per decision on CPU and were **hand-editable**: one added rule took a failure mode from 18.6 to 98.5 percent, and when one tree could not fit a task, two expert trees plus a context-dispatching meta-policy did — mixture-of-experts as the principled version of "split this feature by matchup." The Hearthstone system's per-hero split (+6.5 percent, [EXTRACTED]) says context-splitting keeps paying at industrial scale too.

The case against it. Honesty requires stating this plainly: **the composition is unvalidated.** No verified claim — no paper we found at all — runs this exact loop (hand-authored linear policy, learned value net for credit, expert-iteration weight updates, automatic feature splitting) end-to-end on a card game. The pieces are individually evidenced; the assembly is ours to prove. It is also the path with the most moving parts to build, and its ceiling is bounded by the expressiveness of the rule layer plus whatever the splitting mechanism can grow.

ROAD NOT TAKEN Verdict

Adopted as the backbone. It is the only path that simultaneously fits the compute budget (its expensive part is Path D's verified-feasible value net), collapses manual effort (labels come from value deltas, not humans), preserves the interpretable deliverable (the rule layer remains the runtime policy), and fails gracefully (every stage is independently kill-switched, matching the ship-and-refine doctrine). Its unvalidated-composition risk is managed by the stage gates in Section 6.

4 The five demands, answered mechanism by mechanism

The original brief made five demands of the training system. Here is where each lands in the evidence, and what it becomes in this repository.

4.1 "Context-aware as it fine-tunes weights"

The expert-iteration loop trains weights against per-decision targets **in the context they fired** — the training example is (board state, option menu, expert's choice distribution), not a match-level label. Matchup context enters twice: the Read's archetype is an input to the value net, and the weight table itself can be conditioned per archetype through the existing `Strategy.weight_overrides` carrier — learned now, instead of authored. The Hearthstone per-hero result (+6.5 percent) is the precedent that such splits pay.

4.2 "Able to split features for novel board states and matchups"

Three mechanisms, in escalating order of machinery:

1. **Learned override tables** — the same Hypothesis, different weight per matchup context. Cheapest, already wired, and the first thing the tuner should exploit.
2. **Automatic compound features** (Soemers): when the apprentice's error correlates with two features being co-active, mint their conjunction as a new feature and let the tuner weight it. This is literally "the ML considered splitting a feature because a context demanded it" — with a redundancy penalty so the feature set does not explode. The same paper's warning applies: per-decision feature-evaluation cost ate an eight-fold search slowdown in one game; prune to the top-weight features and measure inference cost per added feature.
3. **Mixture-of-experts dispatch** (INTERPRETER): if one weight table demonstrably cannot cover two regimes, split into expert tables with a small learned gate. Reach for this last; the first two cover most of the demand.

4.3 "Recognise that an agent played every decision perfectly but still lost"

The Suphx delta mechanism, run as an annotator (chess-engine style): for every decision, compute $\Delta = P(\text{win} \mid s, a_{\text{taken}}) - \max_a P(\text{win} \mid s, a)$ under the value net, with the planner's rollout where it is cheap to confirm. A lost match whose deltas are all near zero was played well — file it as luck, update nothing. A won match with a deep negative delta contains a blunder — flag it into the corrections pipeline regardless of the scoreboard. AIVAT [EXTRACTED] is the same idea applied to **evaluation**: strip the variance of the shuffle out of the measurement using the value estimates we already compute. Both directions of the demand — no false blame, no false credit — fall out of one artifact.

4.4 "Know when to take risks, when to pull back and develop"

Mostly, this is what maximising **win probability** rather than material expectation does automatically. A value net trained on terminal win/loss encodes it: when behind, the line

that wins 20 percent of the time beats the line that loses slowly with certainty, and $P(\text{win})$ says so — "playing to outs" is not a personality trait, it is the argmax under the right objective. The literature is thin on explicit risk modulation beyond this (Section 8, question 4), which is itself informative: the strong systems let the win-prob objective carry the burden. Our lethal solver already implements the endgame extreme; the value net extends the same logic backward into the midgame.

4.5 "Adjust strategy mid-match"

Three layers, two of which exist: the Read re-classifies the opponent as evidence arrives and swaps Briefs (built); the Match Planner re-plans as the Threat Clock moves (built); and the value net adds the third — a live $P(\text{win})$ trace whose **level and trend** can gate posture (the desperation gate: when predicted win probability falls below a threshold, loosen the risk constraints on line selection). Suphx's test-time gradient adaptation [EXTRACTED] goes further — fine-tuning the policy during the match on determinised rollouts — but carries real inference cost; park it until the grader budget is measured.

5 The selection framework

5.1 Criteria

Six criteria, drawn directly from the constraint set. A path that fails either of the first two is disqualified outright, whatever its other virtues.

1. **Compute feasibility** — trains on one desktop; infers on CPU inside ten minutes a match.
2. **Manual-effort scaling** — marginal human work per new deck/matchup/rotation approaches doctrine-authoring only.
3. **Competitive ceiling** — headroom to keep climbing the ladder as opponents improve.
4. **Interpretability** — the decision-making approach remains a presentable, debuggable artifact (Strategy category deliverable).
5. **Incrementality and risk** — buildable in kill-switched stages that degrade safely (ship-and-refine doctrine); no big-bang cutover.
6. **Evidence strength** — how much of the path is verified versus extracted versus hoped.

5.2 The matrix

Path	Compute	Effort	Ceiling	Interp.	Increm.	Evidence
A · End-to-end neural policy	Fail	Good	High	Fail	Poor	Verified
B · Regret family (ESCHER)	Unknown	Good	High	Poor	Poor	Partial
C · Sound search (SoG shape)	Infer: Good Train: Fail	Good	High	Fair	Fair	Verified
D · Deep MC value learning	Good	Good	Med	n/a	Good	Verified
E · Replay imitation / offline RL	Good	Good	Med	Poor	Good	Extracted
F · Hybrid rules + learned value	Good	Good	Med–High	Good	Good	Composed

Reading the matrix: A fails two disqualifiers (compute, interpretability). B's compute is unpublished at our scale — a research bet, not a plan. C splits: its inference story is excellent and its training story is out of reach; that is what "upgrade path" means. D is not an agent at all — it is the engine every surviving path shares. E lacks the fuel (a human corpus) to be primary. F composes D's verified engine with mechanisms that are individually evidenced and collectively ours to prove — and it is the only row that is green on all six columns, with its one weakness (composed evidence) explicitly gated in Section 6.

KEY IDEA

The decision, compressed: **train the value function (Path D's verified method), spend it through the rule layer (Path F's architecture), probe it adversarially (Path A's fictitious-play lesson and Path E's replay data), measure it with variance-corrected, stratified protocols (Section 2.6's mandate) — and hold Path C in reserve as the planner's endgame.**

5.3 What would change the answer

A recommendation is only trustworthy with its reversal triggers stated up front:

- **If the Stage 1 value net cannot calibrate** (Brier/log-loss gate fails after honest effort): the keystone is cracked — fall back to expanding the current manual pipeline with better tooling (the annotator idea still works with planner rollouts as a weaker oracle), and revisit Path B's machinery.
- **If serious GPU compute lands** (a lab-scale grant or cluster): Path A's recipe becomes live as a **sparring partner generator** — not as the submitted agent (interpretability still rules) — and Path C's training story reopens.
- **If the annotator's flags disagree wildly with our reviewed-corrections ledger** (Stage 2 gate): the value net understands outcomes but not decisions; insert planner-verified rollouts between net and label, or narrow the annotator to lethal-adjacent decisions where the oracle is exact.
- **If Kaggle exposes deep replay pools of top-ladder agents**: Path E promotes from data channel to co-backbone — imitation-seed the value net and train exploiters directly against reconstructed top opponents.
- **If the ladder reveals we are being exploited** (a specific opponent farms us): the league (Stage 4) jumps the queue — its exploiter probe is the diagnostic and the cure.

6 The recommended program: five stages, five gates

Each stage is independently valuable, kill-switched, and gated — no stage depends on a later one, and every gate is a measurement, not a vibe. Rough effort marks assume the current tooling.

6.1 Stage 0 — The decision-state corpus (enabler, small)

Define one logged record per decision: full engine observation, the option menu as structured actions, Hypothesis signal activations, the Read's archetype posterior, planner verdicts, chosen action, and eventual match outcome. Wire it into `battle.py` self-play at full throughput and into ladder-match telemetry. The cgpy engine twin supplies two things the native engine makes awkward: deterministic seed control (duplicate deals for Stage 5) and a perfect-information tap (oracle-guiding experiments, Suphx-style, where the trainer sees hidden zones the agent cannot).

Gate: a day's self-play run produces a clean corpus a notebook can load; storage cost measured. **Kill risk:** none — pure telemetry.

6.2 Stage 1 — The value network (the keystone, medium)

Train $P(\text{win} \mid \text{state})$ on the corpus, DouZero-style sampled returns first, TD bootstrapping second. Architecture starts embarrassingly small — Modicum won with two hidden layers of 64 — and grows only on gate failure. Inputs: raw state summary plus the Hypothesis activations (our features are a head start, not a liability) plus matchup context. Train across **all deck pairings**, superseding the mirror-only Tier-5 corpus. $\gamma = 1.0$; argmax at any deployment; invalid actions masked structurally.

Gate: calibration (reliability curve, Brier score) on held-out matches from deck pairs unseen in training; must beat the existing Tier-5 model and a logistic baseline on log-loss. **Kill switch:** the net influences nothing at runtime yet — failure costs only the training time.

6.3 Stage 2 — The automated blunder annotator (the effort-collapser, medium)

The chess-annotator loop over every logged match: score all legal options at each decision under the value net (plus planner rollout where cheap), flag decisions where the taken action's $\Delta P(\text{win})$ falls below a threshold, and emit Correction records into the existing pipeline — same schema, automated provenance tag, humans review **flags**, not replays. Calibrate the threshold against the reviewed-corrections ledger: we possess months of human-labeled blunders, which makes this gate unusually sharp.

Gate: on a held-out slice of the ledger, the annotator recovers a majority of human-confirmed blunders at a false-flag rate a weekly review session can absorb. **Payoff when passed:** blunder rounds stop being the bottleneck; every new deck's misplays surface from overnight self-play automatically.

6.4 Stage 3 — Expert-iteration weight tuning (closing the loop, medium-large)

The expert: Turn Planner plus value net, searching shallowly. The apprentice: the linear Hypothesis layer, which remains the runtime policy. Targets: the expert's per-decision choice distributions; updates: cross-entropy SGD (the `tune.py` perceptron generalises). Matchup-conditioned override tables learn first; Soemers-style compound-feature minting switches on second, with the feature-cost meter running (their eight-fold search slowdown warning) and a top-weight pruning pass. Every learned weight change lands as a reviewable diff against the named Hypotheses — interpretability is preserved **as a property of the artifact**, not as a promise.

Gate: the Stage 5 protocol shows non-regression against the hand-tuned baseline across the stratified deal pool, **and** the Stage 2 annotator's flags-per-thousand-decisions falls. **Kill switch:** learned weights ship behind the same override machinery as authored ones — revert is a config flip.

6.5 Stage 4 — The league (robustness, medium)

Population self-play replaces vanilla mirrors: all deck agents, past checkpoints, and a standing **exploiter** — behavior-cloned from the current agent's own logs, PPO-fine-tuned as a best-responder (the exact recipe that exposed ByteRL). Kaggle ladder replays feed behavior-cloned opponent models into the pool. The exploiter's winrate against the main agent becomes a standing dashboard number.

Gate: exploiter winrate below an agreed ceiling; corpus diversity metrics (state-space coverage) improve over mirror-only self-play. **Payoff:** the failure mode that headline win-rates cannot see (Section 2.4) gets an alarm.

6.6 Stage 5 — Evaluation done right (the measurement spine, small-medium)

Replaces "gauntlet is dead" with "gauntlet done right": duplicate deals (same shuffle seeds, both sides — cgpy's determinism makes this cheap); balance over matchup and first/second; **stratify the deal pool by skill-sensitivity** — pre-classify seeds by whether a perfect-information cheating agent beats the baseline on them, and measure on the stratum where decisions matter (the ISMCTS methodology, which found real algorithm differences aggregate winrates completely masked); AIVAT-style correction terms from the value net's own per-decision estimates (an 85 percent standard-deviation cut in the DeepStack evaluation; ten-fold fewer games for significance in poker). The Kaggle ladder remains the final arbiter; this protocol exists so we stop shipping blind between ladder reads.

Gate: a known-difference agent pair (for example, lethal solver on versus off) separates with confidence intervals at a tenth of the raw-winrate sample size.

6.7 The rotation drill

The point of all five stages, rehearsed: when a rotation or meta shift lands, the loop is — deck-genie authors the new doctrine (initial weights as priors, exactly what hand authoring

is **good** at); overnight self-play regenerates the corpus; the value net fine-tunes; expert iteration re-tunes the weights; the annotator surfaces residual gaps; the league's exploiter certifies robustness; the Stage 5 protocol measures the recovery. Human hours in the loop: doctrine authoring, flag review, gate sign-off. That is the effort model the brief demanded.

7 The mathematics of each training element

The five stages above say **what** to build. This section teaches the **math** underneath each, from the ground up — the same beginner-friendly footing as the companion textbook, so you can read every stage as an equation you could compute by hand. Each element also names the runtime tier it feeds, so the training side and the playing side line up.

7.1 The value network — learning $P(\text{win})$ (Stage 1 → T5)

The keystone is a function that reads a board and returns **one number**: the probability we go on to win from here. Start with the smallest honest model, logistic regression. Summarise the board as a list of features $\boldsymbol{\varphi}$ (prize difference, energy on board, the Hypothesis activations we already compute), **standardise** each to mean 0 and spread 1 so no feature dominates by unit choice — call the result $\boldsymbol{z}$ — and push a weighted sum through the logistic squashing function:

$$P(\text{win} \mid \text{board}) = \sigma(\boldsymbol{w} \cdot \boldsymbol{z} + b), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

σ maps any real number into $(0, 1)$: large positive input → near 1, large negative → near 0, zero → exactly 0.5. The weights $\boldsymbol{w}$ and bias b are **learned**.

How? Every state in a self-play game carries a label y: 1 if that game was eventually won, 0 if lost (discount $\gamma = 1$, so a state is worth its final result — no decay inside a match). We choose $\boldsymbol{w}, b$ to make the model's probabilities match those labels, by minimising the **cross-entropy** (log-loss) — the same loss the textbook derives:

$$L = -\frac{1}{N}\sum_{j=1}^{N}\left[y_j \ln P_j + (1 - y_j)\ln(1 - P_j)\right].$$

L is small only when the model is **confident and correct**; it punishes confident-and-wrong harshly (a predicted 0.99 on a lost game costs $-\ln(0.01) \approx 4.6$). Gradient descent walks $\boldsymbol{w}$ downhill on L.

> **KEY IDEA**
> A coin-flip model scores $L = -\ln 0.5 \approx 0.693$ on every state. **Beating** 0.693 **on held-out matches is the whole proof the net learned something real.** That single number is the Stage-1 gate.

The starting architecture is deliberately tiny — Modicum reached master-level poker with two hidden layers of 64 units — and grows only if the gate fails. This trained $P(\text{win})$ **is** the rebuilt Tier-5 value model; the planner adds a capped $W(P - 0.5)$ term to each leaf, so it **refines** the ranking without ever overturning sound combat math.

7.2 Automated blunder labelling — value deltas (Stage 2 → T0 corrections)

Manual blunder rounds do not scale. The value net lets us find blunders automatically. At one of **my** decisions, let $a^\star$ be the option the net rates highest and a the option actually played. The decision's **value drop** is

$$\Delta = P(\text{win} \mid a^\star) - P(\text{win} \mid a) \geq 0.$$

A large Δ means a materially better move was available — a blunder. Flag every decision with $\Delta \geq \theta$ and emit it as a machine-tagged Correction into the **existing** pipeline; humans review the **flags**, never the raw replays.

> **POKÉMON EXAMPLE "played perfectly but lost" — detected, not punished**
> A match we lost, in which **every** one of our decisions had $\Delta \approx 0$ (we always picked at or near the net's best option). That is a game lost to variance or matchup, not to misplay — so it produces **zero** corrections. The old pipeline could not tell this apart from a game full of blunders; value deltas separate "bad play" from "bad luck" for free.

The threshold θ is **calibrated**, not guessed: we already hold months of human-confirmed blunders, so we set θ where the annotator recovers most of them at a false-flag rate a weekly review can absorb — a precision/recall trade you can read straight off the ledger.

7.3 Expert iteration — teaching the rules from a stronger teacher (Stage 3 → T0, T4.4)

Now close the loop. Two players: the **apprentice** is the linear rule layer (T0), which stays the runtime policy; the **expert** is the Turn Planner plus the value net, doing a one-step lookahead the apprentice cannot afford at runtime. For a logged decision the expert scores each legal option i by simulating it and reading the net on the result — with a **sound override**: if the engine says the line wins, its value is 1, not the net's estimate.

$$V_i = \begin{cases} 1 & \text{if the line is an engine-verified win} \\ 0 & \text{if it loses} \\ P(\text{win} \mid \text{board after option } i) & \text{otherwise} \end{cases}.$$

The expert's preferred option is $\arg\max_i V_i$. Wherever the expert's best beats the apprentice's pick by a margin, $V_{a^\star} - V_a \geq \theta$, we emit a Correction — **the blunder annotator and the expert are literally the same detector**. Those Corrections drive the very same weight fitter the textbook builds: the soft-margin perceptron. On a violated ranking constraint it nudges the weights toward the better option,

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta(\boldsymbol{\varphi}_{a^\star} - \boldsymbol{\varphi}_a),$$

with a pull-to-seed regulariser and a **pocket** that keeps the best weights seen. Two extensions: matchup-conditioned tables learn a per-archetype adjustment $\delta_A[h]$ (this is exactly

runtime sub-tier T4.4), and compound features are minted from co-active pairs — behind a cost meter, because per-node feature evaluation can throttle the search.

> **KEY IDEA**
> No new training machinery is invented: the expert **writes corrections**, and the existing perceptron **consumes** them. The value net turns a scarce human resource (labelled blunders) into an abundant automated one, and the interpretable rule layer stays the thing that actually plays.

7.4 The league and exploiters — why a high win-rate can lie (Stage 4)

Beating a pool of opponents 90% of the time does **not** prove you are hard to beat. A dedicated **best-responder** — an opponent trained only to exploit **your** fixed policy — can still crush you. Formally, an agent's **exploitability** is how much a best response beats it:

$$\text{exploitability}(\pi) = \max_{\pi'} \text{winrate}(\pi' \text{ vs } \pi) - \frac{1}{2},$$

the gap between the best counter-strategy's win-rate and an even match. Headline win-rate against a **fixed** pool never measures this; only a probe that **adapts to you** does. So the league keeps a standing **exploiter**: behaviour-clone the current agent from its own logs, then fine-tune it purely to beat that agent. Its win-rate against the main agent is a dashboard alarm — when it spikes, we have a blind spot no gauntlet would have shown.

7.5 Evaluation done right — squeezing signal from noisy games (Stage 5)

Raw win-rate is a terribly noisy ruler. For a measured win-rate $\hat{p}$ over N games the 95% confidence half-width is

$$1.96\sqrt{\frac{\hat{p}(1-\hat{p})}{N}} \quad \propto \quad \frac{1}{\sqrt{N}},$$

so **halving** the interval needs 4 × the games and **quartering** it needs 16 ×. Three tricks buy that back:

1. **Paired deltas.** Rather than compare two agents' absolute win-rates, measure the same deck with a change **on** versus **off** against the **same** opponent, winrate(D@on) − winrate(D@off). The raw deck matchup cancels, leaving only the effect of the change — a much lower-variance quantity.
2. **Duplicate deals + stratification.** Replay the **same** shuffle under both contestants (luck cancels), and measure on the **skill-sensitive** subset of deals — those a perfect-information cheater wins — where decisions actually decide the game. Aggregate win-rate hides real differences that show up sharply on this stratum.
3. **AIVAT (a control variate).** Subtract a baseline that tracks the outcome but averages to zero — the value net's own per-decision $P(\text{win})$ estimates serve for free. Subtracting

a correlated, mean-zero quantity leaves the average unchanged but **cuts the variance**: about an 85% standard-deviation reduction in the DeepStack evaluation, roughly a ten-fold cut in games needed for significance.

FOR THIS REPO Evaluation reuses the net we already built

Every one of these tricks either **reuses** the value net (AIVAT baselines, the cheating-agent stratifier) or **reuses** existing paired-A/B stats. Evaluation is not a new subsystem so much as the disciplined application of the value net we already had to build — which is why Stage 5 is the cheapest stage, not the most expensive. The Kaggle ladder stays the final arbiter; this protocol just stops us shipping blind between ladder reads.

8 Risks, named

1. **Self-play bias in, garbage out.** The value net only knows states our own policy visits; a systematic blind spot in the Pilot becomes a blind spot in the oracle that is supposed to correct the Pilot. Mitigations: league diversity (Stage 4 exists partly for this), exploration temperature in corpus generation, oracle-guiding experiments through cgpy's perfect-information tap, and the ladder-replay distribution check.
2. **Annotator threshold pathology.** Too loose and we drown in flags (the manual bottleneck returns wearing a costume); too tight and real blunders pass. The reviewed-corrections ledger is the calibration set; the false-flag budget is set by what a weekly review absorbs, and the threshold is a dial, not a constant.
3. **Distribution shift at the ladder.** Kaggle opponents are not our league. The Metamon evidence cuts both ways — their agents generalised to human ladders, but they trained on human data. Watch the live $P(\text{win})$ trace's calibration against actual ladder outcomes; drift there is the early warning.
4. **Feature bloat and inference cost.** Compound-feature minting can quietly eat the CPU budget (the Soemers eight-fold warning). Every minted feature carries a measured per-decision cost; the pruning pass is not optional.
5. **The unvalidated composition.** Stated once more without cushioning: no published system runs this exact assembly. The stage gates are the containment — each stage is useful alone, and the first two stages (corpus, value net, annotator) are valuable even if Stage 3 never ships.
6. **Native-engine nondeterminism.** Duplicate-deal evaluation and replay-exact annotation both want seed control the native engine may not expose; cgpy parity (46 of 46 in the cross-engine audit at last count) is the workaround and must stay maintained.

9 Open questions

1. **Sample sizes for our effect sizes.** With stratification and AIVAT-style correction, how many duplicate-dealt games detect a two-to-three-point winrate delta at 95 percent confidence? The Hearthstone bar was 45,000 raw; poker's correction bought a factor of ten. Stage 5′s gate experiment answers this empirically in an afternoon of simulator time.
2. **Does regret-based learning fit a desktop at TCG scale?** ESCHER's variance wins were measured on small benchmarks with oracle values. If Stage 3 stalls, this is the alternative to price out — possibly on a reduced game (fixed decks, no sideboard decisions) first.
3. **The interpretability-preserving loop end-to-end.** Our composition is novel. If it works, it is — noted without irony — a publishable result and a strong Strategy-category writeup on its own.
4. **Risk modulation beyond win-probability maximisation.** Distributional value heads, explicit variance preferences when behind, opponent-model-conditioned aggression: thin literature. Park until the live $P(\text{win})$ trace exists, then look at whether measured desperation gates beat the implicit behaviour.
5. **Minimal viable league.** AlphaStar's league was industrial. What does the three-agents-and-an-exploiter version buy? Stage 4′s corpus-diversity gate is the cheap first read.

10 Source annex

Twenty-one primary sources fetched; 103 claims extracted; the top 25 adversarially verified (three refutation votes each): 24 confirmed, one refuted. The refuted claim — that CCG hidden-information spaces make search and subgame-resolving methods **categorically unusable** — died zero votes to three, and its death is load-bearing for Path C's status as a live upgrade path.

Recency caveats, so nothing here ossifies: DouZero (2021) has been surpassed on its own leaderboard by PerfectDou and successors, and Suphx (2020) by Tencent's LuckyJ — the method lessons hold, the rankings do not. The Hearthstone human evaluation is a single player on a restricted 2015 card pool. The reward-shaping negative result is one study on a deterministic, simplified CCG with small networks.

System / paper	What it contributes here	Status
DouZero (ICML 2021) arXiv 2106.06135	Desktop-budget Deep Monte-Carlo; action-as-input encoding	Verified
PerfectDou (NeurIPS 2022) arXiv 2203.16406	Confirms action-as-input under PPO	Verified
Suphx (MSR 2020) arXiv 2003.13590	Global reward predictor; delta credit; oracle guiding; test-time adaptation	Verified
VR-MCCFR (AAAI 2019) arXiv 1809.03057	Baselines cut variance 1000×, stay unbiased	Verified
DREAM (2020) arXiv 2006.10410	Neural history baselines; the importance-sampling flaw	Verified
ESCHER (ICLR 2023) arXiv 2206.04122	Removes importance sampling via history value function	Verified
ByteDance Hearthstone (IEEE CoG 2023) arXiv 2303.05197	Top-human result; compute bill; 45k-match protocol; gauntlet mispredicts humans; per-hero split	Verified
ByteRL / LOCM (2023) arXiv 2303.04096	OSFP population training; champion agent	Verified
ByteRL exploitability (2024) arXiv 2404.16689	BC + PPO best-responder beats the champion 90%	Verified
Desktop PPO for LOCM (Ent. Computing 2023) PDF	Desktop plateau; reward-shaping harm; set-encoder and masking levers	Verified
Student of Games (Science Advances 2023) arXiv 2112.03178	Sound search shape; enumeration bottleneck; sampling workaround	Verified

PyTAG (2024) arXiv 2405.18123	Independent gauntlet-unreliability corroboration	Verified
Residual Policy Learning (2018) arXiv 1812.06298	Learn a residual on a hand-designed controller	Extracted
Biasing MCTS with Features (2019) arXiv 1903.08942	Expert iteration for linear feature policies; automatic feature growth; feature-cost warning	Extracted
Evolving LOCM evaluators (2021) arXiv 2105.01115	Linear beats trees at low compute; self-play fitness wins	Extracted
VIPER (NeurIPS 2018) PDF	Q-loss-weighted distillation; verifiable small trees	Extracted
INTERPRETER (2024) arXiv 2405.14956	Editable distilled programs; MoE splitting; 79 µs CPU inference	Extracted
SCoBots (2024) arXiv 2401.05821	Concept bottlenecks surface policy defects	Extracted
AlphaStar (Nature 2019) Nature	League with exploiters; 3M non-transitive cycles; ladder as final measure	Extracted
AIVAT (2017) + Schmid thesis (2021) arXiv 1612.06915 · 2111.05884	Unbiased variance-cut evaluation; naive self-play divergence; abstraction obsolescence	Extracted
ISMCTS (IEEE TCIAIG 2012) PDF	Determinization pathologies; deal-stratified evaluation	Extracted
Modicum (NeurIPS 2018) arXiv 1805.08195	Master play on a 4-core CPU; multi-valued leaves; tiny value nets	Extracted
Metamon (RLC 2025) arXiv 2504.04395	Replay reconstruction; imitation → offline RL → self-play; top-10% human ladder	Extracted

---

Provenance. Generated from the 2026-07-11 deep-research workflow run (`wf_3bda6aa0-d3b`): five search angles, 103 agents, 21 sources fetched, 103 claims extracted, 25 verified by three-vote adversarial refutation. The machine-readable findings live in `docs/research/ml-training-system.md`; this document adds the path comparison, selection framework, and staged program. Companion reading inside the repo: `docs/tuning/methodology.md` (the manual loop this program automates), `docs/architecture/tiers.md` (where each stage lands), and ADR-0042/ADR-0050.